

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;

public class GradeAnalyzer {

    public static void main(String[] args) {

        final String filename = "scores.txt";
        final String outputFileName = "report.txt";
        // testCalculateAverage();
        Integer[] lineCount = { 0 };
        final ArrayList<Integer> scores = readScores(filename,
lineCount);
        final Double average = calculateAverage(scores);
        Integer highestScore = Integer.MIN_VALUE;
        Integer lowestScore = Integer.MAX_VALUE;
        for (int i = 0; i < scores.size(); i++) {
            int score = scores.get(i);
            if (score > highestScore) {
                highestScore = scores.get(i);
            }
            if (score < lowestScore) {
                lowestScore = scores.get(i);
            }
        }
        writeReport(scores, average, highestScore, lowestScore,
lineCount[0], outputFileName);

    }

    public static ArrayList<Integer> readScores(String filename,
Integer[] lineCount) {
        ArrayList<Integer> scores = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new
FileReader(filename))) {
            String line;
            while ((line = reader.readLine()) != null) {
                lineCount[0]++;
                String trimmedLine = line.trim();
                if (trimmedLine.isEmpty()) {
                    System.out.println("Skipping empty line !");
                }
                try {
                    int score = Integer.parseInt(trimmedLine);
                    scores.add(score);
                }
            }
        }
    }
}

```

```

        } catch (NumberFormatException e) {
            System.out.println("Skipping invalid score: " +
line);
        }

    }

    } catch (IOException e) {
        e.printStackTrace();
        System.out.println("Error in reading file !");
    }
    return scores;
}

// Returns the average of a list of scores, or 0.0 if the list is
empty
public static double calculateAverage(ArrayList<Integer> scores) {

    if (scores.isEmpty()) {
        return 0.0;
    }
    double sum = 0.0;
    for (int i = 0; i < scores.size(); i++) {
        sum += scores.get(i);
    }
    return sum / scores.size();
}

// Writes and prints the report
public static void writeReport(ArrayList<Integer> scores,
    double avg, int high, int low, int lineCount,
    String outputFile) {
    int countA = 0, countB = 0, countC = 0, countD = 0, countF =
0;
    for (int i = 0; i < scores.size(); i++) {
        int score = scores.get(i);
        // A = 90+, B = 80 to 89, C = 70 to 79, D = 60 to 69, F =
below 60.
        if (score >= 90) {
            countA++;
        } else if (score >= 80) {
            countB++;
        } else if (score >= 70) {
            countC++;
        } else if (score >= 60) {
            countD++;
        } else {
            countF++;

```

```

    }
}
try (BufferedWriter writer = new BufferedWriter(new
FileWriter(outputFile))) {
    String out = "=== Grade Analysis Report ===";
    writer.write(out);
    writer.newLine();
    System.out.println(out);
    writer.newLine();

    out = String.format("Total scores processed: %d%n",
lineCount);
    writer.write(out);
    writer.newLine();
    System.out.println(out);

    out = String.format("Invalid lines skipped:  %d%n",
lineCount = scores.size());
    writer.write(out);
    writer.newLine();
    System.out.println(out);

    out = String.format("Average score:  %.2f%n", avg);
    writer.write(out);
    writer.newLine();
    System.out.println(out);

    out = String.format("Highest score: %d%n", high);
    writer.write(out);
    writer.newLine();
    System.out.println(out);

    out = String.format("Lowest score:  %d%n", low);
    writer.write(out);
    writer.newLine();
    System.out.println(out);

    out = "Grade distribution:";
    writer.write(out);
    writer.newLine();
    writer.newLine();
    System.out.println(out);

    out = String.format("A (90-100):      %d%n", countA);
    writer.write(out);
    writer.newLine();
    System.out.println(out);

    out = String.format("B (80-89):      %d%n", countB);
    writer.write(out);

```

```

        writer.newLine();
        System.out.println(out);

        out = String.format("C (70-79):      %d%n", countC);
        writer.write(out);
        writer.newLine();
        System.out.println(out);

        out = String.format("D (60-69):      %d%n", countD);
        writer.write(out);
        writer.newLine();
        System.out.println(out);

        out = String.format("F (below 60):   %d%n", countF);
        writer.write(out);
        System.out.println(out);
    } catch (IOException e) {
        System.out.println("Could not write file: " +
e.getMessage());
    }
}

public void testCalculateAverage() {
    ArrayList<Integer> testScores = new ArrayList();
    testScores.add(87);
    testScores.add(75);
    testScores.add(50);

    double avarage = calculateAverage(testScores);
    System.out.println(avarage);
}
}

```